

- 1 Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
root@DUDDUKURIs:~/OS_Skill/Week-10$ echo "Hello from Charan" > original.txt
root@DUDDUKURIs:~/OS_Skill/Week-10$ cat original.txt
Hello from Charan
root@DUDDUKURIs:~/OS_Skill/Week-10$ ls -li original.txt
13732 original.txt
root@DUDDUKURIs:~/OS_Skill/Week-10$ stat original.txt
  File: original.txt
  Size: 20          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 13732         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/charan)   Gid: ( 1000/charan)
Access: 2026-09-12 10:13:14.617607567 +0000
Modify: 2026-09-12 10:12:56.021606597 +0000
Change: 2026-09-12 10:12:56.021606597 +0000
 Birth: 2026-09-12 10:12:56.021606597 +0000
root@DUDDUKURIs:~/OS_Skill/Week-10$ ln original.txt hardlink.txt
root@DUDDUKURIs:~/OS_Skill/Week-10$ ls -li original.txt hardlink.txt
13732 -rw-r--r--  2 charan charan 20 Sep 12 10:12 hardlink.txt
13732 -rw-r--r--  2 charan charan 20 Sep 12 10:12 original.txt
root@DUDDUKURIs:~/OS_Skill/Week-10$ echo "Modified using hard link" >> hardlink.txt
root@DUDDUKURIs:~/OS_Skill/Week-10$ cat original.txt
Hello from Charan
Modified using hard link
root@DUDDUKURIs:~/OS_Skill/Week-10$ stat original.txt
  File: original.txt
  Size: 45          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 13732         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/charan)   Gid: ( 1000/charan)
Access: 2026-09-12 10:14:34.025609030 +0000
Modify: 2026-09-12 10:14:19.201608803 +0000
Change: 2026-09-12 10:14:19.201608803 +0000
 Birth: 2026-09-12 10:12:56.021606597 +0000
root@DUDDUKURIs:~/OS_Skill/Week-10$ ln -s original.txt symlink.txt
root@DUDDUKURIs:~/OS_Skill/Week-10$ ls -li original.txt hardlink.txt symlink.txt
13732 -rw-r--r--  2 charan charan 45 Sep 12 10:14 hardlink.txt
13732 -rw-r--r--  2 charan charan 45 Sep 12 10:14 original.txt
13745 lrwxrwxrwx  1 charan charan 12 Sep 12 10:15 symlink.txt -> original.txt
root@DUDDUKURIs:~/OS_Skill/Week-10$ ls -li original.txt
13732 original.txt
root@DUDDUKURIs:~/OS_Skill/Week-10$ find . -inum 13732
./original.txt
./hardlink.txt
root@DUDDUKURIs:~/OS_Skill/Week-10$ rm original.txt
root@DUDDUKURIs:~/OS_Skill/Week-10$ cat hardlink.txt
Hello from Charan
Modified using hard link
```

- 2 Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```
root@DUDDUKURIs  ×  +  ∨

GNU nano 8.7.1 src/mmap_f
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main() {
    int fd;
    struct stat st;
    char *data;

    fd = open("mmap_input.txt", O_RDWR);

    if (fd < 0) {
        perror("open");
        return 1;
    }

    if (fstat(fd, &st) < 0) {
        perror("fstat");
        close(fd);
        return 1;
    }

    data = mmap(NULL, st.st_size,
                PROT_READ | PROT_WRITE,
                MAP_SHARED, fd, 0);

    if (data == MAP_FAILED) {
        perror("mmap");
        close(fd);
        return 1;
    }

    printf("Original file content:\n%s\n", data);

    strcpy(data, "Modified using mmap!");

    printf("File modified successfully.\n");

    msync(data, st.st_size, MS_SYNC);

    munmap(data, st.st_size);
    close(fd);

    return 0;
}
```

Output:

```
root@DUDDUKURIs:~/OS_Skill/Week-10$ echo "Hello from mmap! This file will be modified." > mmap_input.txt
root@DUDDUKURIs:~/OS_Skill/Week-10$ cat mmap_input.txt
Hello from mmap! This file will be modified.
root@DUDDUKURIs:~/OS_Skill/Week-10$ nano src/mmap_file.c

root@DUDDUKURIs:~/OS_Skill/Week-10$ gcc src/mmap_file.c -o bin/mmap_file
root@DUDDUKURIs:~/OS_Skill/Week-10$ ./bin/mmap_file
Original file content:
Hello from mmap! This file will be modified.

File modified successfully.
```